

基於大型語言模型的家庭網路資安自動化健檢系統

LLM-Powered Home Network Security Auditor

完整開發與說明報告

組別:第 168 組

組員姓名:陳俊瑋、陳揚盛

學號: 陳俊瑋 112502038、陳揚盛 112502527

項目	內容
組別	第 168 組
組員姓名	陳俊瑋、陳揚盛
學號	陳俊瑋 112502038、陳揚盛 112502527
課程主題	大型語言模型與資訊安全系統
文件日期	2026 年 6 月 9 日

本報告依據專案 README、system-design、proposal、demo script、regression log 與程式碼模組整理。

1. 專案摘要

本專案開發一套以 LLM Agent 為核心的家庭網路資安健檢系統。使用者只需在 Streamlit 介面按下開始掃描，系統便會自動取得本機網路資訊、Wi-Fi 安全狀態、區

域網路裝置與開放服務，並透過 RAG 知識庫比對 CVE 與 OWASP IoT Top 10 等安全知識，最後產出 A 到 F 等級的結構化安全報告與可追問的自然語言 Q&A。

系統定位並非取代專業滲透測試，而是降低家庭與小型網路使用者的資安門檻，將掃描器、漏洞資料庫與風險分級轉換成可閱讀、可執行的修復建議。

面向	說明
核心目標	自動化 LAN 探測、裝置盤點、漏洞查詢、風險分級、修復建議與後續問答。
主要使用者	缺乏資安背景但希望了解家中 Wi-Fi、路由器與 IoT 裝置風險的一般使用者。
主要成果	完成掃描工具層、RAG 檢索層、LLM Agent、報告產生器、Streamlit UI、Demo 情境與 Regression 測試。
驗證方式	使用 golden scan、prompt probes、Q&A regression、單元測試與三組 demo 報告驗證功能。

2. 研究背景與動機

智慧家庭設備普及後，家庭網路不再只有電腦與手機，也包含 IP Camera、智慧音箱、智慧電視、掃地機器人、NAS、路由器管理服務等多種連網裝置。這些設備常因預設密碼、過時韌體、開放不必要服務、缺少網路隔離而成為攻擊入口。

傳統工具如 Nmap、NVD、OWASP 指南雖然資料完整，但輸出偏技術導向。一般使用者即使取得掃描結果，也未必能判斷哪個問題最嚴重、是否真的影響自己、以及應該先修哪一項。因此，本專案以 LLM Agent 串接掃描工具與 RAG 知識庫，整合資料蒐集、風險推理與可讀報告。

3. 專案目標與需求

- 自動化探測：在使用者授權的 LAN 內，自動取得本機 IP、子網路、閘道、DNS、Wi-Fi 加密與活躍裝置。
- 攻擊面盤點：使用 Nmap 掃描常見連接埠與服務版本，標記 gateway、IoT 類設備、管理介面與遠端服務。
- 漏洞與知識查詢：使用 RAG 對 CVE、OWASP IoT Top 10 與內建安全文件進行語意檢索。
- 風險分級：將 findings 分成 high、medium、low、info，並彙整為 A、B、C、D、F 的總體安全等級。
- 自然語言報告與 Q&A：產出 Markdown/Streamlit 報告，並允許使用者以中文或英文追問掃描結果與修復步驟。
- 可重現 Demo：提供 clean network、risky IoT、vulnerable router 三種固定情境，降低展示時對真實 LAN 狀態的依賴。

4. 系統整體架構

系統採分層式架構：UI 層負責操作與呈現，Agent 層負責流程編排與安全推理，Scanners 層負責實際環境探測，RAG 層負責知識檢索，Models 層提供跨模組一致的資料結構。

層級	模組	責任
前端介面層	app.py、ui/chat.py、ui/report_view.py	提供 Streamlit 操作介面、掃描按鈕、Demo 載入、報告視覺化與 Q&A 對話。
LLM Agent 層	agent/core.py、agent/tools.py、agent/reporter.py	執行掃描流程、封裝工具呼叫、綜合 CVE 與 port risk、產生 ScanReport。
工具執行層	scanners/*	取得網路資訊、Wi-Fi 狀態、Nmap 裝置掃描、路由器

		HTTP/UPnP 探測與開放 port baseline 風險。
RAG 知識庫層	rag/*、 data/knowledge_base、 data/vector_db	使用 Azure Embedding 與 ChromaDB 建立 CVE/KB 檢索能力，支援 product/version 與 port risk 查詢。
資料模型層	models/*	以 Pydantic 定義 Device、Port、NetworkInfo、WiFIInfo、RouterInfo、CVE、RiskFinding、ScanReport。
測試與腳本	scripts/*、tests/*	建立知識庫、匯出 Demo、執行 golden scan、prompt probes、Q&A regression 與單元測試。

4.1 主要資料流

- 使用者在 Streamlit 介面點擊 Start Scan 或載入 Demo scenario。
- SecurityAgent 依序取得 NetworkInfo、WiFIInfo、Device 清單與 RouterInfo。
- Agent 針對 router model、device vendor/product/service 呼叫 lookup_cve，並對開放連接埠呼叫 check_open_ports_risk。
- Agent 根據掃描結果、CVE context、KB snippets 與 prompt 規則合成 RiskFinding。
- reporter.assemble_report 排序 findings、計算 severity counts、套用 grade rubric，並輸出 ScanReport 與 Markdown summary。
- UI 將 report 呈現成 header cards、network summary、risk dimensions、devices table、findings tabs 與 Markdown report；Q&A 使用目前 report 作為 grounded context。

5. 開發環境與技術堆疊

類別	採用技術	用途
程式語言	Python 3.12	主要後端、掃描、RAG、Agent 與 UI 實作。
前端	Streamlit	快速建立互動式掃描與報告介面。
LLM/Agent	Azure OpenAI、LangChain Tool-Calling	自然語言推理、工具編排、報告合成與 Q&A。
Embedding	text-embedding-3-small	將 CVE 與安全文件轉為向量。
向量資料庫	ChromaDB	儲存與查詢 CVE chunks 與 KB chunks。
網路探測	Nmap、python-nmap、psutil、requests、系統 Wi-Fi 工具	裝置發現、port/service 掃描、Wi-Fi 與路由器探測。
資料驗證	Pydantic	統一跨模組資料結構並降低格式錯誤。
測試	pytest、fixture scripts	驗證 parser、retriever、reporter、prompt grounding 與 demo scenarios。

6. 核心資料模型設計

資料模型放在 `models/`，讓掃描工具、Agent、報告與 UI 使用一致欄位，避免各層以鬆散 dict 互傳造成錯誤。

模型	重點欄位	用途
Port	port、protocol、service、product、version、state	描述開放服務與版本資訊，是 port risk 與 CVE 查詢的重要來源。
Device	ip、hostname、mac、vendor、os、role、open_ports	表示 LAN 中一台裝置，支援 gateway 與 IoT 類型推理。
NetworkInfo	local_ip、subnet、gateway、dns_servers、interface、is_wireless	描述本機網路視角與掃描範圍。
WiFiInfo	ssid、encryption、band、signal_dbm	評估 Wi-Fi 加密與無線狀態。
RouterInfo	ip、model、firmware_version、admin_interface、upnp_enabled、remote_management_open	聚合路由器特定探測結果。
CVE	cve_id、description、cvss_score、published、affected_products、source	RAG 檢索後回傳的漏洞資料。
RiskFinding	severity、dimension、title、description、affected、recommendation、cves	報告中的單一風險項目。

ScanReport	generated_at、network、wifi、router、devices、findings、overall_grade、summary_markdown	UI、Markdown 報告與 Q&A 的核心輸出物件。
------------	--	------------------------------

7. 掃描工具層實作

7.1 網路資訊探測

`scanners/network_info.py` 以 `psutil` 與系統指令取得 IPv4 介面、預設路由、DNS 與網段。它針對 Linux、macOS、Windows 提供不同 parser，並在 `psutil` 不可用時以 `ipconfig`、`route`、`ifconfig` 等輸出做 fallback。

7.2 Wi-Fi 安全探測

`scanners/wifi_security.py` 支援 Linux `nmcli/iwconfig`、macOS `system_profiler/wdutil/airport` 與 Windows `netsh`。解析結果會正規化成 WPA3、WPA2、WPA、WEP、Open 等加密模式，並嘗試轉換訊號強度與頻段。

7.3 Nmap 區域網路掃描

`scanners/nmap_scanner.py` 透過 `python-nmap` 掃描指定 subnet。一般模式掃描 Top 100 ports 並執行服務版本偵測；若程式以 `root/sudo` 執行，可啟用 OS detection 與 ARP discovery，得到更完整的裝置資訊。掃描結果會轉成 Device 與 Port 模型，並使用 OUI lookup 補足廠牌資訊。

7.4 路由器探測

`scanners/router_probe.py` 對 gateway 嘗試 HTTP/HTTPS probe，解析 Server header、HTML title、model hint、firmware version，並檢查常見管理介面、UPnP 與遠端管理相關 port。此設計能在不登入路由器的前提下取得可公開觀察的弱訊號。

7.5 Port Risk Baseline

scanners/port_risk.py 對每個 open port 建立 baseline finding。管理服務如 SSH、Telnet、FTP、RDP、VNC 與 IoT 服務如 RTSP、UPnP、MQTT、CoAP 會被特別標記，再搭配 RAG 查到的 port risk 文件補充說明與建議。

8. RAG 知識庫設計

RAG 層由 embeddings.py、vector_store.py、retriever.py 與 ingest loaders 組成。專案狀態紀錄顯示知識庫已完成 end-to-end 建置，包含 5,355 個 CVE chunks 與 200 個 KB chunks，並驗證 semantic retrieval 與 metadata-filtered retrieval。

元件	設計重點
nvd_loader.py	呼叫 NVD API 2.0，解析 CVE id、英文描述、CVSS 分數、vector、受影響 CPE 與發布時間。
owasp_loader.py	載入 Markdown、HTML、PDF 等靜態安全文件，並保留 source/path metadata。
chunker.py	以 chunk size 與 overlap 切分長文件，避免 embedding context 過長並保留語意連續性。
embeddings.py	封裝 Azure OpenAI Embedding client，包含 batch 與 retry。
vector_store.py	封裝 ChromaDB collection add/query/count，並將 query 結果整理為一致格式。
retriever.py	提供 lookup_cve(product, version, k, min_cvss)、lookup_port_risk(port, service, k)、search(query, k)。

9. LLM Agent 與 Prompt 設計

SecurityAgent 是系統的大腦，負責把 deterministic scanner 與 probabilistic LLM 推理接在一起。run_full_scan 會依序執行 network、Wi-Fi、Nmap、router、CVE gathering、port risk 與 finding synthesis，最後交給 reporter 組裝報告。

9.1 工具封裝

agent/tools.py 將 scanner 與 retriever 包裝成 LangChain tool，包含 get_network_info、get_wifi_security、scan_network、check_router_info、lookup_cve、check_open_ports_risk。每個 tool 使用 Pydantic args schema，並將結果序列化成 JSON，方便 LLM 與程式端共同處理。

9.2 Grounding 與反幻覺策略

- Prompt 明確要求 CVE 只能引用提供的 context，不得自行補造 CVE id。
- 若產品家族相同但版本未知，高 CVSS CVE 不直接升為 high，而以 possible/verify firmware version 的 medium finding 呈現。
- 固定常見 home network severity：open/WEP 為 high、WPA2 without WPA3 為 medium、UPnP on 為 medium、LAN-only admin panel 為 low、缺資料為 info。
- Q&A 使用目前 ScanReport 與 retriever snippets，回答需回到掃描事實，遇到報告沒有的內容要明確說無法確認。

10. 報告產生與評分規則

agent/reporter.py 負責將 findings 排序、統計、分級並渲染 Markdown。severity 順序為 high、medium、low、info；報告包含 header、network summary、device inventory、findings、risk dimension breakdown 與 remediation。

等級	UI 標籤	判定意義

A	Secure	沒有會降低分數的問題，或僅呈現安全資訊。
B	Good	有 low 或少量 medium finding，但未達高風險。
C	Needs improvement	出現 high finding，或 medium finding 數量較多，需要改善。
D	At risk	多個 high finding，網路已存在明顯高風險。
F	Critical	高風險問題達 critical 門檻，應立即處理。

Risk dimensions 分為 Router vulnerability、Wi-Fi encryption、IoT exposure、Network isolation、Remote attack surface。UI 會依維度顯示最嚴重等級與 finding 數量，協助使用者快速理解問題集中在哪裡。

11. 前端介面與使用流程

Streamlit UI 由 `ui/chat.py` 與 `ui/report_view.py` 組成。首頁提供 Start Scan、Demo scenario、Load Demo Report 與 Reset。掃描期間會顯示階段式進度文字；報告完成後，畫面呈現 Security Report header、Network、Risk Dimensions、Devices、Findings tabs、Markdown report 與 Follow-up Q&A。

流程	使用者操作	系統反應
Live Scan	點擊 Start Scan	建立 agent，執行真實 LAN 掃描，完成後顯示報告並啟用 Q&A。
Demo Report	選擇 scenario 並點擊 Load Demo Report	載入固定 ScanReport，用於課堂展示與截圖。

Follow-up Q&A	輸入自然語言問題	根據目前報告與 KB context 串流回答。
Reset	點擊 Reset	清除目前 report、agent session 與 Q&A history。

12. Demo 情境與代表性輸出

Scenario	代表情境	預期等級與重點
Clean network	乾淨家庭網路，WPA3、兩台裝置、無 findings。	Grade A；驗證系統不會在無風險時硬造問題。
Risky IoT	小型 LAN，有 gateway FTP、RTSP/tcpwrapped 等需確認服務。	Grade B；2 devices、2 medium、4 info，可展示中文 Q&A。
Vulnerable router	含 BusyBox router vulnerability 與實際 CVE 引用。	Grade D；High findings，報告引用 CVE-2018-5371 並給修復建議。

Demo 腳本建議依序展示首頁、running scan、Risky IoT 報告摘要、risk dimensions/devices/findings、中文追問，再切換 Clean network 與 Vulnerable router 展示不同分級。

13. 測試與驗證

專案同時使用 deterministic 與 LLM-based regression。deterministic 測試保證資料模型、scanner parser、reporter rubric 與 demo fixture 穩定；LLM-based 測試則用 temperature 0 的 prompt probes 與 Q&A regression 檢查 grounding。

測試項目	覆蓋內容	紀錄結果
pytest tests/test_scanners.py	OUI lookup、Wi-Fi parser、router probe、Nmap mapping、port risk baseline。	涵蓋跨平台 parser 與 scanner output normalization。
tests/test_demo_scenarios.py	三個 demo scenario 的 grade 與 vulnerable router CVE。	確認 demo 報告可重現課堂展示。
golden_scan.py	固定 golden fixtures 經 assemble_report 後的 grade、severity mix、排序、CVE、五大 dimensions、remediation 與 A-F rubric。	2026-06-05 紀錄：25/25 pass。
prompt_probes.py	空 CVE、相關 CVE、不相干 CVE、版本未知 CVE、缺資料等情境。	更新後紀錄：14/14 invariants hold。
qa_regression.py	固定問題集測試回答是否引用真實 report facts 並避免 fabricated facts。	golden report 紀錄：8/8 pass。

Regression 紀錄也保留 prompt convergence 過程：早期 admin panel reachable 的 severity 可能在 low/info 之間漂移，後續透過 prompt anchor 固定為 LAN-only admin panel low；版本未知 CVE 則從 confident high 調整為 medium 並要求使用者確認 firmware version。

14. 分工與開發歷程

成員	主要負責模組	具體工作	
陳揚盛	掃描工具、前端、測試與 Demo	Nmap 掃描封裝、Wi-Fi 探測、路由器 probe、Streamlit UI、demo scenarios、截圖與展示流程。	
陳俊瑋	Agent、Prompt、RAG 與報告邏輯	LangChain Agent、system/report/QA prompts、RAG ingestion/retrieval、reporter、grading rubric、prompt regression。	
階段	工作內容		完成狀態
Phase 0	專案 bootstrap 與基本結構建立。		Done
Phase 1	scanner foundation 與 RAG knowledge base。		Done
Phase 2	retriever API 與 scanner enrichment，凍結 lookup_cve/lookup_port_risk/search contract。		Done
Phase 3	port_risk.py、tools.py、prompts draft 等 cross-cutting glue。		Done
Phase 4	Agent executor、reporter 與 UI shell。		Done

Phase 5	真實 LAN end-to-end 與 UI screenshots 。	Done
Phase 6	Regression 、polish 、demo reports 與 acceptance checks 。	Done

15. 安裝、執行與操作說明

15.1 環境準備

- 安裝 Python 3.12 與 uv，建立虛擬環境：`uv venv && uv pip install -r requirements.txt`。
- 安裝 Nmap；若需要 OS detection，可用 `sudo` 或系統管理員權限執行。
- 建立 `.env`，填入 Azure OpenAI chat deployment 與 text-embedding-3-small embedding 設定。
- 執行 `scripts/build_kb.py` 建立 RAG knowledge base。

15.2 執行方式

目的	指令
啟動 Streamlit UI	<code>streamlit run app.py</code>
CLI 掃描與互動 Q&A	<code>.venv/bin/python scripts/run_scan.py</code>
從 cached report 離線展示	<code>.venv/bin/python scripts/run_scan.py --offline report.json</code>
匯出 demo reports	<code>python scripts/demo_scenarios.py --export - -out docs/demo-reports</code>

執行 deterministic regression	<code>.venv/bin/python scripts/golden_scan.py</code>
-----------------------------	--

16. 倫理、安全與限制

- 授權原則：本系統僅應掃描自己擁有或明確獲授權的網路，不得用於未授權目標。
- 掃描範圍：目前設計主要評估 LAN 內部攻擊面，不能完整代表外網暴露狀態。
- 資料隱私：LLM 推理會使用掃描摘要與裝置/服務資訊，部署時需注意雲端 API 的資料處理政策。
- 辨識限制：Nmap OS detection、HTTP banner 與 router model hint 可能不完整或不準確。
- 知識庫限制：CVE 結果取決於 KB 更新頻率、產品/版本資訊完整度與檢索命中品質。
- LLM 限制：即使有 prompt grounding，安全建議仍應由使用者或資安人員確認後再執行。

17. 專案成果與結論

本專案完成了從掃描、檢索、推理、報告到互動問答的端到端家庭網路資安健檢流程。技術上，系統展示 LLM Agent 可以作為資安工具的 orchestration layer，將 Nmap、Wi-Fi parser、router probe、RAG 檢索與報告產生整合為單一使用者體驗。

在可信度方面，專案沒有只依賴 LLM 自由生成，而是透過 Pydantic models、deterministic scanner、RAG context、prompt anchors、golden fixtures 與 regression scripts 限制輸出範圍。這讓系統能在家庭網路場景中產生較穩定、可追溯且對非專業使用者友善的建議。

未來可延伸方向包含：加入外網暴露檢查、支援更多 router vendor fingerprint、定期自動更新 CVE KB、提供修復前後差異比較、輸出 PDF/Word 報告，以及加入更細緻的 IoT segmentation 檢查。

18. 參考資料

- NIST National Vulnerability Database (NVD): <https://nvd.nist.gov/>
- OWASP Internet of Things Project / IoT Top 10: <https://owasp.org/www-project-internet-of-things/>
- Nmap Reference Guide: <https://nmap.org/book/man.html>
- LangChain Documentation: <https://python.langchain.com/docs/>
- ChromaDB Documentation: <https://docs.trychroma.com/>
- Streamlit Documentation: <https://docs.streamlit.io/>
- Lewis et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, NeurIPS 2020.
- Yao et al., ReAct: Synergizing Reasoning and Acting in Language Models, ICLR 2023.

附錄 A. 專案主要檔案對照

路徑	說明
README.md	專案架構、資料流、環境需求與快速開始。
docs/system-design.md	開發階段、完成狀態、Acceptance 與 UI/報告規格。
docs/proposal.md	研究背景、目標、架構與原始分工規劃。
docs/demo-script.md	Phase 6 demo 流程、截圖 checklist 與 scenario talk track。
docs/phase6-regression.md	Regression 與 prompt convergence 紀錄。

agent/core.py	SecurityAgent、完整掃描流程、CVE gathering、findings synthesis 與 Q&A。
agent/reporter.py	ScanReport 組裝、grading rubric、Markdown 渲染與 remediation 排序。
scanners/	network_info、wifi_security、nmap_scanner、router_probe、port_risk。
rag/	embedding client、ChromaDB wrapper、retriever API、NVD/OWASP ingestion。
ui/	Streamlit chat shell 與 report visualization。
scripts/	KB build、demo reports、golden scan、prompt probes、Q&A regression。
tests/	單元測試與 demo scenario validation。